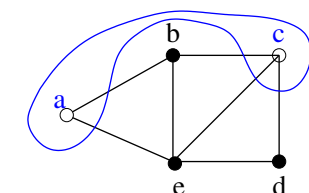


3 Algorismes d'aproximació

3.1. El problema del Tall Maxim (MAX-CUT). Donat un graf no dirigit $G = (V, E)$, el problema del maximum cut (MAX-CUT) es trobar la partició $S \cup \bar{S}$ de V tal que maximitze el nombre d'arestes entre S i $\bar{S}$ ($C(S, \bar{S})$), on $S \subseteq V$ i $\bar{S} = V - S$. La maximització entre totes les possibles particions de V . El problema del MAX-CUT és NP-hard en general. Donat

Per Alg approx cal:

- Correcció (do una solució)
- Coste
- Tasa d'aproximació



$S = \{a, c\}$ $\bar{S} = \{b, e, d\}$

$C(S, \bar{S}) = \{(ab), (ae), (bc), (ce), (cd)\}$

Per veure tota, pensar en que volent tan pa pa

Si $C = a + b$

$\max(a, b) \geq \frac{C}{2}$

Figura 1: Exemple de MAX-CUT, amb cost òptim 5

$G = (V, E)$ considereu el següent algorisme golafre (greedy) pel problema del MAX-CUT: Ordeneu els vèrtexs en ordre decreixent respecte al seu grau (nombre veïns). Considereu el resultat de l'ordenació s'escriu com a $(v_1, v_2, \dots, v_n)$.

Inicialment tenim $S = \emptyset = \bar{S}$. Al primer pas $v_1 \in S$. Al pas i -èsim col·loquem v_i a S o $\bar{S}$ de manera que maximitze $|C(S, \bar{S})|$.

- Demostreu la correctesa i doneu la complexitat de l'algorisme golafre.
- Demostreu que aquest algorisme golafre és una 2-aproximació al MAX-CUT

a) Correcte, perquè la solució dona el set S .

Cost $O(n \log n + m)$ on $n = |V|$, per la ordenació.

$O(n + m)$ Ordenació lineal, counting sort! $0 \leq d(u) \leq n-1$

b) Punt per per, no afegint vèrtexs.

$$c = a + b$$

Afegirem $g_{\max_i}$

$$\max(a, b) \geq \frac{c}{2}$$

$$G = \sum g_i \quad G = c \quad g_{\max_i} \geq \frac{G}{n}$$

E_i son vèrtexs para i

$\{E_i\}$ é una partició d' E

Para i arribar al cost $\geq \frac{|E_i|}{2}$ vèrtexs

$$S_n \quad d = a + b + c, \quad \max(a+c, b+c) \geq \frac{d}{2}$$

3.2. Consider the following algorithms that have as input a boolean formula in 3-CNF

Algorithm A

- (a) Let c_0 be the number of true clauses when all variables are set to value 0.
- (b) Let c_1 be the number of true clauses when all variables are set to value 1.
- (c) return the maximum among c_0 and c_1 .

Is there a constant r for which the algorithm is a r -approximation for MaxSat?

2-approx

Correcció: és correcte perquè retorna el màxim possible calculat.

Cost: Cost lineal en mida de la fórmula $O(|F|)$

Taxa aproximada: $d = c_0 + c_1 + (n - (c_0 + c_1))$

$$c_0 = b + c$$

$$c_1 = a + c \Rightarrow \max(a, b) \geq \frac{m}{2}$$

3.3. Consider the following algorithm:

```

function EDGES( $G$ :graph)
   $U = \emptyset$ ;  $E = E(G)$ ;
  while  $E \neq \emptyset$  do
    select any edge  $e = (u, v) \in E$ ;
     $U = U \cup \{u, v\}$ ;
     $E = E - \{e' \in E \mid e' \text{ incident to } e\}$ 
  return  $U$ 

```

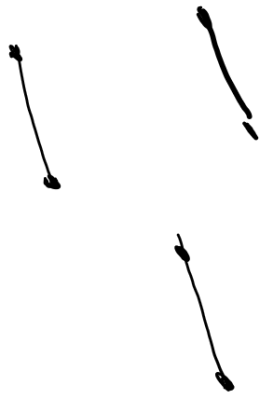
subreix vèrtex, cobreix tot el costat

Prove that Edges is a 2-approximation for the Minimum vertex cover problem.

Covered: És cobert perquè cobrim tot el costat per definició, però eliminem vèrtex amb un extrem cobert

Cost: $O(m+n)$ (recorrem vèrtex i eliminem veïns)

Aprox: Per definició de l'algorisme avem vèrtex Matchings



Però si fos òptim, no agafaria els dos vèrtexs, només un:



(cobrim
extrem)

3.4. Consideramos el siguiente escenario: tenemos un conjunto de n ciudades con distancias mínimas entre ellas (verifican la desigualdad triangular). Queremos seleccionar un subconjunto C de k ciudades en las que queremos ubicar un centro comercial. Asumiendo que las personas que viven en una ciudad comprarán en el centro comercial más próximo se quiere buscar una ubicación C de manera que todas las ciudades tengan un centro comercial a distancia menor que r en la que intentamos minimizar r sin perder cobertura. Para ello diremos que C es un r -recubrimiento si todas las ciudades están a distancia como mucho r de una ciudad en C . Sea $r(C)$ el mínimo r para el que C es un r -recubrimiento. Nuestro objetivo es encontrar C con k vértices para el que $r(C)$ es mínimo.

(a) Demuestra que si $k \geq n$, la solución formada por todas las ciudades es óptima.

A partir de ahora asumiremos que $k \leq n$.

(b) Suponiendo que S es el conjunto de ciudades, considera el siguiente algoritmo:

```

Seleccionar cualquier ciudad  $s \in S$  y define  $C = \{s\}$ 
while  $|C| \neq k$  do
    seleccionar una ciudad  $s \in S$  que maximiza la distancia de  $s$  a  $C$ ;
     $C = C \cup \{s\}$ 
return  $C$ 

```

Demuestra que es un algoritmo de aproximación polinómico con tasa de aproximación 2.

Correcció:

Complexitat:

n -aproximació:

3.5. Planificació.

Tenim un conjunt de n tasques. La tasca i es descriu per un parell $T_i = (s_i, d_i)$ on s_i es el seu temps de disponibilitat i d_i la seva duració. L'objectiu es planificar totes les tasques en un processador de manera que: (a) cap tasca s'executa abans del seu temps de disponibilitat, (b) les tasques s'executen sense interrupció, i (d) el temps de finalització del procesament de totes les tasques sigui mínim.

Se sap que, quan és possible interrompre qualsevol tasca en execució i reiniciar-le més tard des del punt d'aturada, l'algorisme voraç que executa en cada punt de temps la tasca (disponible) més propera al seu temps de finalització, aconsegueix el desitjat mínim. Anomenarem a aquest algorisme Voraç1.

Considereu el procediment següent:

- (a) Executa Voraç1. $\rightarrow$ dona el mínim, però pot interrompre tasques.
- (b) Ordena les tasques en l'ordre ascendent del seu temps de finalització d'acord amb la solució proporcionada per versió Voraç1. $\rightarrow$ ordena segon temps fin
- (c) Les tasques s'executen en aquest ordre i sense interrupció, afegint el temps d'espera necessari fins que la tasca estigui disponible.

Demostreu que aquest algorisme es una 2-aproximació pel problema plantejat.

• Correcció: l'algorisme propant dona una solució al problema, ja que aquest s'assegura que les tasques no siguin interrompudes i que comencin a partir de s_i per a tota T_i .

• Complexitat: Suposant que Voraç1 costa $O(1)$. L'ordenar les tasques té cost $O(T \log T)$. Recorre les tasques té cost $O(T)$. Per tant el cost és $O(T \log T)$.

• 2-aproximació:

Voraç1 en dona seq de tasques a imprimir:



Algorisme retorna: T_2, T_1, T_3 (ordre d'ocorrença / (el fin a punt de)

$T_1: (0, 5) \Rightarrow \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ T_1 & T_1 & T_2 & T_2 & T_1 & T_1 & T_1 \end{matrix} \Rightarrow T_2, T_1$

$T_2: (2, 2)$

$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ T_2 & T_2 & T_1 & T_1 & T_1 & T_1 & T_1 & T_1 \end{matrix}$

Per a veure la 2-approximació:

Sigui $C_{\max}^*$ el temps òptim calculat per varoz1 , demostrare que:

$$(\text{temp } \text{varoz1}) C_{\max}^* \leq 2 \cdot C_{\max}^* (\text{temp } \text{algorisme opax})$$

Sigui B l'últim bloc contigu que fa el nostre algorisme.

Sigui t on comença, i sigui k la primera tasca que es fa.

Com abans estava inactiu i k és la primera tasca que fa tenim que $t = s_k$. Obviament s_k és posterior o igual a quan acaba l'algorisme, $s_k \leq C_{\max}^*$.

Sigui $\sum_{i \in B} d_i$ la duració total del bloc B .

Per lògica $\sum_{i \in B} d_i \leq C_{\max}^*$.

El temps total serà l'espera fins a s_k més $\sum_{i \in B} d_i$:

$$C_{\max}^* = t + \sum d_i$$

|||

$$C_{\max}^* \leq C_{\max}^* + C_{\max}^*$$

$$C_{\max}^* \leq 2 C_{\max}^*$$

- 3.6. Donat com a entrada un graf no dirigit $G = (V, E)$, definim el problema de GST (graf sense triangles) com el problema de seleccionar el màxim nombre d'arestes E , de manera que el graf $G' = (V, E')$ no contingui cap triangle (i.e. no hi han 3 vèrtexs x, y, z tal que $(x, y), (x, z)$ i (y, z) siguin arestes a G'). Aquest problema és NP-hard.

Donat $G = (V, E)$ considereu el següent algorisme golafre (greedy):

Ordeneu els vèrtexs en ordre decreixent respecte al seu grau (nombre veïns). Considereu el resultat de l'ordenació s'escriu com a $(v_1, v_2, \dots, v_n)$.

Inicialment tenim $S = \emptyset = \bar{S}$. Al primer pas $v_1 \in S$. Al pas i -èsim col·loquem v_i a S o $\bar{S}$ de manera que maximitze $|C(S, \bar{S})|$. Prenem com $E' = \{(u, v) \mid u \in S, v \notin S\}$

Aquest algorisme, és una 2-aproximació polinòmica al problema?.

Correcció: És correcció perquè calula un Max-Cut i es queda amb el tall, llavors no hi ha triangle. Així maximitzem arestes.

Cost: $O(n+m)$

Taxa aprox:

Trivial amb ex 1. //

$$\max(a, b) \geq \sqrt{a \cdot b}$$

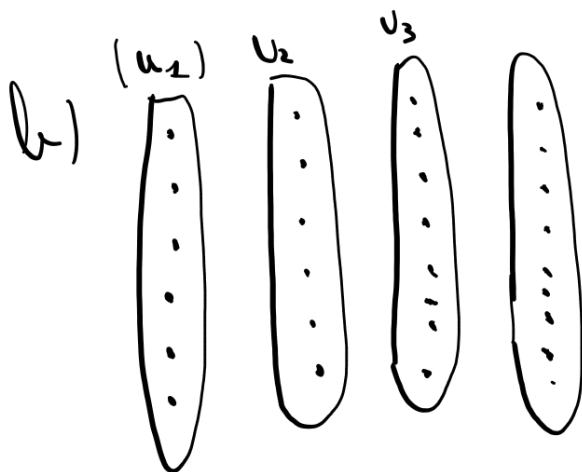
3.7. Consider the Max Clique problem. Given an undirected graph $G = (V, E)$ compute a set of vertices that induce a complete subgraph with maximum size.

For each $k \geq 1$, define G^k to be the undirected graph (V^k, E^k) , where V^k is the set of all ordered k -tuples of vertices from V . E^k is defined so that $(v_1, \dots, v_k)$ is adjacent to $(u_1, \dots, u_k)$ iff, for $1 \leq i \leq k$, either $(v_i, u_i) \in E$ or $v_i = u_i$.

- (a) Prove that the size of a maximum size clique in G^k is the k -th power of the corresponding size in G .
- (b) Argue that if there is a constant approximation algorithm for Max Clique, then there is a polynomial time approximation schema for the problem.

$$V^k = \left\{ \begin{array}{l} (1, 2, 3) \quad (2, 1, 3) \quad (1, 1, 1) \quad (2, 2, 2) \\ (1, 1, 2) \end{array} \right\} \quad \left. \begin{array}{l} k=3 \\ n^k \end{array} \right\}$$

a) Per combinatorics, there are n^k combinations, so that is complex.



$$\max_{1 \leq i \leq k} |D_i| \geq \sqrt[k]{|D|}$$

$$|D| = \prod_{1 \leq i \leq k} |D_i|$$

$$\text{opt}(G) \geq \sqrt[k]{\text{opt}(G^k)}$$

- 3.8. Consider the MAXIMUM COVERAGE problem: Given sets $S_1, \dots, S_m$ over a universe of elements $U = \{1, \dots, n\} = \cup_{i=1}^n S_i$ and a positive integer k . Choose k sets that cover as many elements as possible, that is

$$opt(x) = \max\{|\cup_{i \in I} S_i| \mid |I| = k\}.$$

Provide a greedy approximation algorithm with rate $\frac{e}{e-1} \approx 1.58$.

- 3.9. The Barcelona Diagonal University campus is shared among the UPC, UB, CSIC, and other public and private entities. As a result there are many surveillance cameras placed outside the buildings whose images can be accessed by different surveillance companies and in some cases by different surveillance teams inside the company. Each camera has a unique surveillance team that can visualize the images on the spot. The UPC Chancellor needs to establish a surveillance plan to be set during the next Chancellor elections. The aim of the plan is to guarantee that the voting is done without pressure.

To establish the plan the UPC Committee has set a collection of *surveillance locations* that require visual surveillance. For each of those locations, they have collected the surveillance teams that have cameras visualizing the location. Fortunately all the surveillance locations can be covered by an already existing camera, so the task can be performed without additional infrastructure.

Due to the economical situation, the UPC Chancellor wants to expend the minimum possible amount of money. The administrative office has negotiated and agreed the surveillance cost with each of the surveillance teams. Now they are seeking a solution that allows to keep surveillance on all the surveillance locations with minimum total cost. Assume, for this exercise, that a surveillance team receives a fix amount of money for looking to the images of all the cameras assigned to it.

Consider the algorithm that repeatedly chooses the team that cover the maximum number of still uncovered locations.

Is this algorithm a constant factor approximation for the problem? Justify its correctness and efficiency.

- 3.10. The association for the promotion of the European Identity is planning a workshop formed by a series of debates over different European topics. They have a *participant's* list formed by the persons that have committed to participate in the workshop provided the association issues them a formal invitation.

The organizing committee in view of the planned topics and previous experiences has selected for each debate two disjoint lists of people: the *success* list and the *failure* list. The committee is considering those list in an optimistic perspective. The presence of at least one of the persons in the success list or the absence of at least one of the persons in the failure list of a particular debate guarantees that this debate will be successful.

The organizing committee faces the problem of selecting the subset of people to whom the association will send a formal invitation. The association wish to invite a set of people that guarantees that the number of successful debates (according to the previous rule) is maximized.

Design a 2-approximation algorithm for the problem. Justify its correctness and efficiency.

- 3.11. Considereu una formula booleana sobre un conjunt de n variables en 3-CNF, totes les clausules tenen 3 literals diferents. Una 3-MCNF és una formula en 3-CNF on, a més, cap literal és la negació d'una variable.

Per suposat una 3-MCNF sempre admet una assignació de valors a les variables que la satisfà. Per això considerem el problema min-3-MCNF què donada una formula booleana F en 3-MCNF ha de trobar una assignació de valor a les variables que satisfaci la formula amb el nombre més petit possible de variables a 1.

- (a) Considereu l'algorisme següent:

```
1: procedure G-3-MCNF( $F$ )
2:    $F$  en 3-MCNF amb clausules  $\mathcal{C} = \{C_1, \dots, C_m\}$  over variables  $x_1, \dots, x_n$ 
3:    $x_i = false$ , per  $1 \leq i \leq n$ 
4:   while  $\mathcal{C} \neq \emptyset$  do
5:     Extreu una clausula  $C$  de  $\mathcal{C}$ 
6:     Sigui  $x_i$  una de les variables a  $C$ 
7:      $x_i = true$ 
8:     Treu de  $\mathcal{C}$  totes les clausules que continguin  $x_i$ 
9:   Return  $x$ 
```

És G-3-MCNF un algorisme d'aproximació constant per min-3-MCNF?

- (b) Doneu una 3-aproximació per min-3-MCNF.